DevOps Capstone Project

WashQ

Booking washing machines in a hostel,
so two people never get the same one.

Amrinder Singh

24BCS10596

`github.com/aamrindersingh/washq`

The problem

Sunday evening in any hostel

Every machine is taken, someone's washing has been sitting in one for two hours, and the machine everyone avoids has been broken for a month without anyone reporting it.

- You walk down three times to check if anything is free
- Two people put their clothes in the same machine and argue
- Nobody reports the broken one, so it stays broken

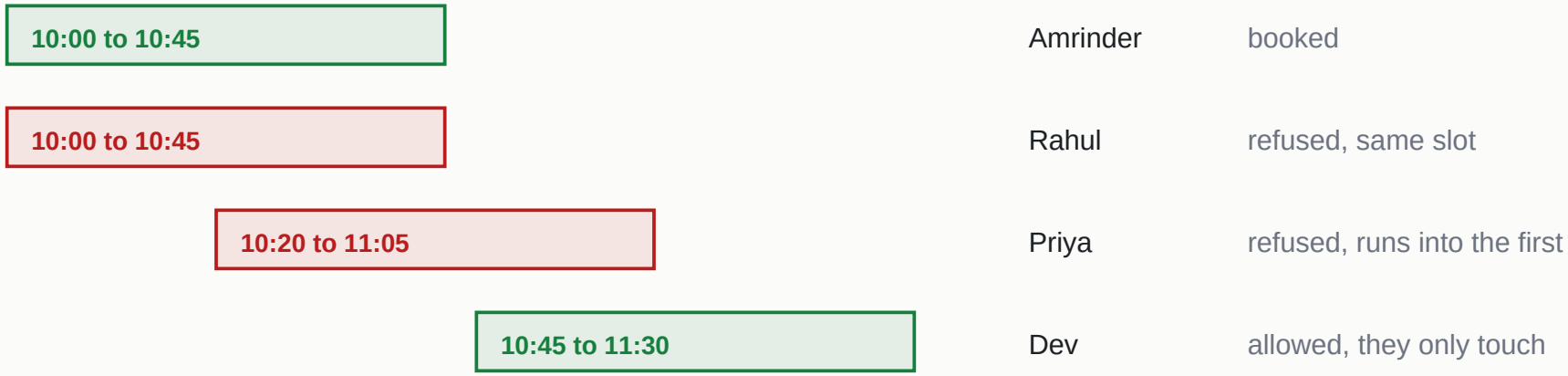
WashQ is a board of every machine with its live status, and a 45 minute slot you can claim.

The app is small on purpose. Two tables and one real rule, so the work is in everything around it.

The one rule

A machine is never booked twice

Two bookings clash when each one starts before the other ends.



Amrinder	booked
Rahul	refused, same slot
Priya	refused, runs into the first
Dev	allowed, they only touch

Touching edges are not an overlap, so back to back washes are fine. Cancelling puts the slot back.

Most of the 22 tests are about these four cases.

Stack

What it is built from

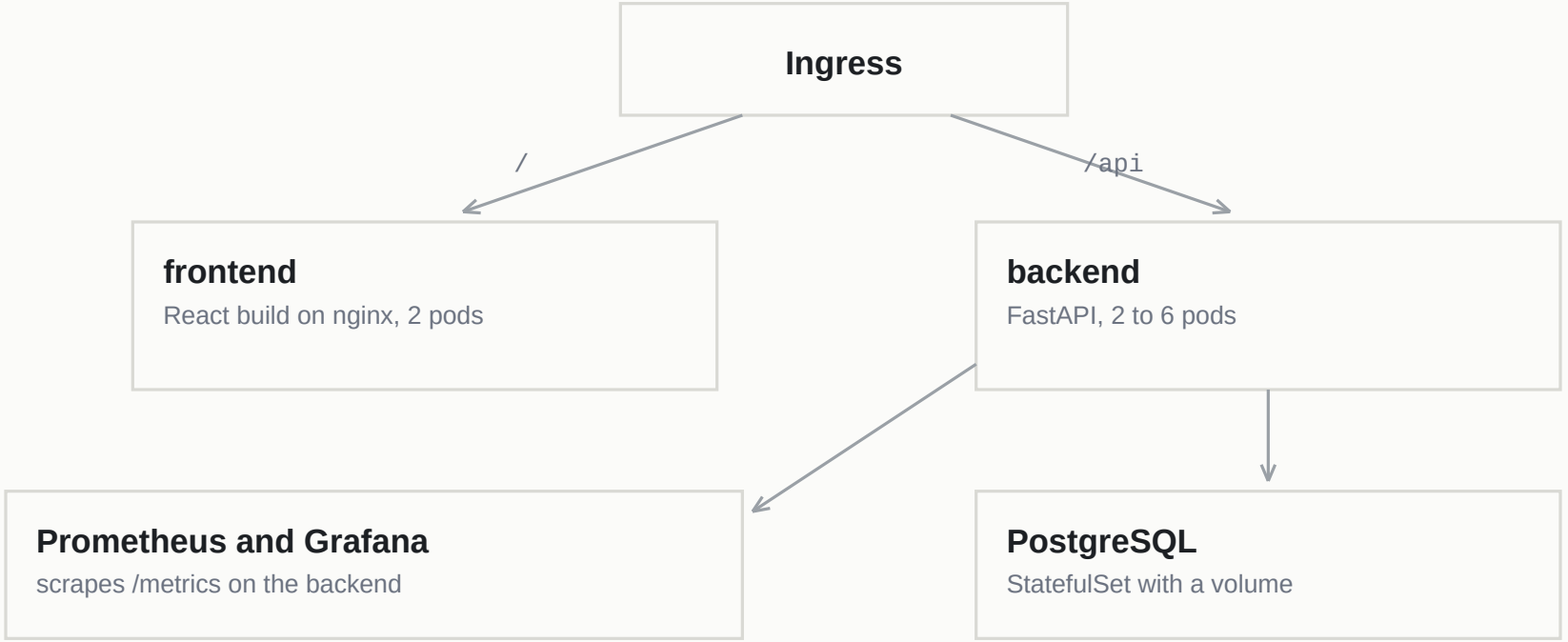
Frontend	React 18, Vite, served by nginx
Backend	Python 3.12, FastAPI, SQLAlchemy 2
Database	PostgreSQL 16, Alembic migrations
Tests	pytest, 22 of them, on SQLite
Containers	Docker, multi-stage frontend build

Pipeline	GitHub Actions, images to GHCR
Security	Trivy, blocks HIGH and CRITICAL
Infra	Terraform, AWS VPC and EKS
Cluster	Kubernetes, Helm, Ingress, HPA
Monitoring	Prometheus and Grafana
Deploys	Argo CD, so a deploy is a commit

Architecture

How it fits together

One ingress in front, so the browser only ever talks to one origin and there is no CORS in production.



Twelve endpoints

The brief asks for four. GET, POST, PUT and DELETE are all used.

GET	/health
GET	/health/ready
GET	/api/machines
POST	/api/machines
PUT	/api/machines/{id}/status
DELETE	/api/machines/{id}
GET	/api/bookings
POST	/api/bookings
PUT	/api/bookings/{id}
DELETE	/api/bookings/{id}
GET	/api/stats
GET	/metrics

```
$ curl -X POST localhost:8000/api/bookings -d '{"machine_id":1, ...}'
{"detail":"W1 is taken until 10:45"}
```

Testing

22 tests, and they gate everything

A failing test stops the pipeline before an image is even built.

- The same slot twice is refused
- A slot starting partway through another is refused
- Back to back slots are allowed
- Cancelling frees the slot so it can be booked again
- Rescheduling onto a taken slot is refused

> **They never touch the real database**
>
> conftest.py points DATABASE_URL at a temporary SQLite file before app.db is imported, and overrides get_db. So no Postgres is needed to run them.

```
$ pytest -v
========== 22 passed in 0.21s ==========
```

Two images, neither running as root

frontend, two stages

node:22-alpine builds the bundle, nginx:1.27-alpine serves it. No Node in the final image. nginx runs as its own user on 8080, which meant moving the pid file to /tmp.

backend

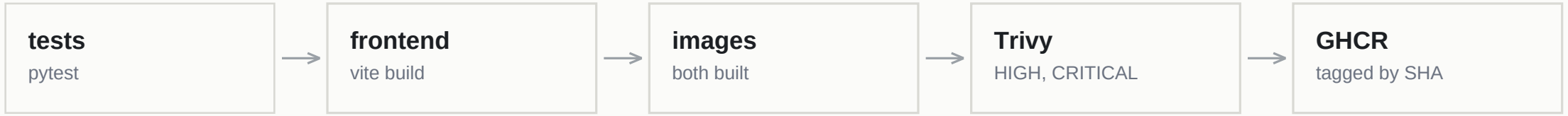
python:3.12-slim, running as uid 10001. Requirements are copied before the source so the dependency layer caches until they actually change.

The whole stack is one command:

```
$ docker compose up --build
postgres   Up (healthy)   0.0.0.0:5432->5432
backend    Up (healthy)   0.0.0.0:8000->8000
frontend   Up (healthy)   0.0.0.0:3000->8080
```

Pipeline

Every push to main



- Tests and the frontend build run in parallel and gate everything after them
- Images are built locally first, so Trivy scans exactly what would be published
- The push only happens if the scan passes
- Tagged with the commit SHA. No latest tag, so a deploy names one exact build
- Built for amd64 and arm64, so it runs on EKS and on a laptop

```
ghcr.io/aamrindersingh/washq/backend:<commit sha>
ghcr.io/aamrindersingh/washq/frontend:<commit sha>
```

Security

Trivy found something real

The scan is a gate. The job fails and nothing gets pushed.

First scan of the frontend

43 HIGH or CRITICAL findings, every one of them with a fix already available. The worst was CVE-2026-31789, a CRITICAL in libcrypto3, which is OpenSSL.

Why

nginx:1.27-alpine carries whatever Alpine packages were current the day that tag was built, and OpenSSL had been patched since. A pinned tag is not a safe tag.

One line fixed it:

```
RUN apk upgrade --no-cache

rescan:  0 HIGH or CRITICAL with a fix available
```

The backend shows 44 findings, all unfixed upstream. We cannot patch what nobody has patched, so the gate ignores unfixed CVEs and blocks everything that has a fix.

Infrastructure

Terraform, not console clicks

VPC	10.20.0.0/16, DNS on
Subnets	two public, in two availability zones
Gateway	internet gateway, route table, associations
EKS	Kubernetes 1.31
Nodes	managed group, 2 to 4 m7i-flex.large
Addons	vpc-cni, kube-proxy, coredns, metrics-server, EBS CSI
IAM	control plane, nodes, and an IRSA role for the EBS driver

```
$ terraform plan
Plan: 24 to add, 0 to change,
      0 to destroy.
```

Two easy things to miss

The subnets need the elb and cluster tags or a LoadBalancer never gets an address. And metrics-server has to be added, because EKS leaves it out and the HPA reads CPU from it.

terraform destroy runs after the demo. Nothing is left on the bill.

Kubernetes

One chart, two value files

- A Deployment and a ClusterIP Service for the backend and for the frontend
- Postgres as a StatefulSet with a volume, so a restart keeps the bookings
- Ingress sending /api and /metrics to the backend, everything else to the app
- An HPA on backend CPU, and a ServiceMonitor so Prometheus finds it

Migrations first

An init container runs alembic upgrade head before the API takes traffic, so it never serves against an old schema.

Two health endpoints

Liveness uses /health, which skips the database. Readiness uses /health/ready, which does not. If both hit the database a slow query would kill healthy pods.

No replicas field

With autoscaling on, the chart leaves replicas unset. If Helm set it too, every upgrade would reset the count and fight the HPA.

Autoscaling

The Sunday rush, on demand

The backend scales on CPU between 2 and 6 pods.

```
$ ./scripts/load-test.sh http://washq.local 60 20

# before
washq-backend   cpu: 2%/70%     MIN 2  MAX 6  REPLICAS 2

# during
washq-backend   cpu: 93%/70%    MIN 2  MAX 6  REPLICAS 4
```

Scale down waits 120 seconds. Without that the pod count flaps every time traffic dips for a moment.

Monitoring

Metrics the app actually cares about

Request rate, latency and status codes come free from the instrumentator. These five are the app's own:

`washq_bookings_created_total`	slots booked
`washq_double_booking_rejected_total`	clashes refused, the rule in numbers
`washq_bookings_cancelled_total`	bookings cancelled
`washq_machines_busy`	machines washing right now
`washq_machines_out_of_order`	machines reported broken

Prometheus

Finds the backend through a ServiceMonitor in the chart. The stack chart only watches its own by default, so that had to be overridden.

Grafana

The dashboard loads from a labelled ConfigMap, so there is nothing to import by hand. Eight panels, all with data.

Deploys

A deploy is a commit

Argo CD watches the repo and makes the cluster match it. Nobody runs helm against the cluster.

1 push to main → 2 pipeline builds, scans and pushes → 3 bump the tag in the repo → 4 Argo rolls it out

selfHeal

Anything changed by hand on the cluster gets put back. Which also means editing a live Deployment to debug something does not stick.

ignoreDifferences

On the replica count, because the HPA owns it. Without it Argo treats the scaled count as drift and resets it every few minutes.

```
washq   Synced   Healthy
```

What went wrong

Four things that broke while building it

None of these came from a tutorial. They came from building it.

Trivy would have been a lie

43 fixable CVEs in the frontend image. The easy move was to loosen the gate. apk upgrade took it to zero instead.

A six hour hang

The arm64 image ran npm under emulation and hung until GitHub killed the job at its limit. Pinning the build stage to the native platform took it to 73 seconds.

Argo never went Synced

Kubernetes stamps extra fields onto volumeClaimTemplates after creation. Nothing was broken, Argo was just reading defaults as drift.

A 500 I caused myself

I wiped Postgres under running pods. Migrations run at pod start, so they kept talking to an empty database until I restarted them.

Real AWS

What only real AWS showed

A local cluster hid every one of these. They only appeared on EKS.

t3.medium is not allowed on a Free plan account

The cluster built, then the node group sat in CREATE_FAILED for 33 minutes. Switched to m7i-flex.large from the list of eligible types.

EKS has no disk driver out of the box

It ships a gp2 StorageClass but not the EBS CSI driver behind it. Postgres stayed Pending and the backend crash looped, which looked like a backend bug and was not.

The driver needed its own IAM identity

The node role was not enough, the controller cannot reach instance metadata. IRSA gives it a role scoped to one service account.

No load balancers on a Free plan

OperationNotPermitted. So the ingress is reached by port forward, and on a paid account the same Ingress gets an ELB with no changes.

Destroy left a disk behind

Kubernetes made the Postgres volume, so Terraform never knew about it. Found it billing afterwards and deleted it.

Limits

What this is not

- Postgres is one pod. The volume keeps the data, but there is no replication or backup
- No login. Anyone who can open the page can book as anyone
- The clash check is in application code, not a database constraint
- No HTTPS. A real cluster would run cert-manager and a real hostname
- One environment. Two value files, but a single cluster

What I would do next

The database constraint first, because it is the only one here that could actually lose a booking rather than just annoy someone. Postgres can enforce it with an exclusion constraint on a time range, and the index is already there for it. Then a migration Job instead of an init container, then logins.